BCC_S32K144_FreeMASTER

Example project for Battery Cell Controller SW Driver
for MC33771C and MC33772C

Date: *15 December 2020*

Revision: *2.2*

Overview

The purpose of this example project is to demonstrate several Battery Cell Controller functions with use of the Battery Cell Controller SW Driver and S32K144 microcontroller. The project uses FreeMASTER tool to control and display status of MC33771C BCC device.

Hardware Requirements

Following is required:

- S32K144EVB-Q100 (MCU board)
- BATT-14CEMULATOR (Battery pack emulator)
- USB Micro B cable
- Communication type related EVBs:
    - SPI communication: FRDM33771CSPEVB (BCC board)
    - TPL communication:
        - FRDMDUAL33664EVB (Board with two isolated network transceivers)
        - RD33771CDSTEVB (BCC board)

Setting up Hardware

1. Make sure that you **disconnect** SBC (System Basis Chip) and power your S32K144EVB-Q100 from OpenSDA USB instead. SBC initialization and setting is not in scope of this example.

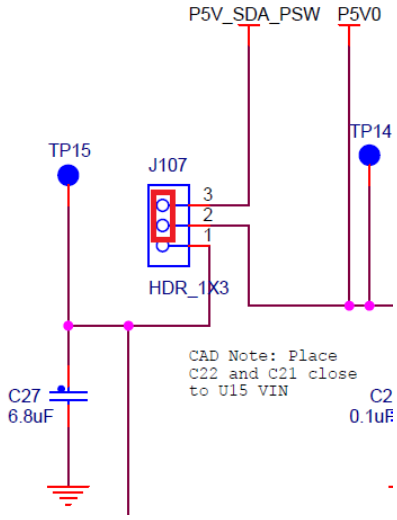

Figure 1. S32K144EVB-Q100 power source selection

2. **For SPI setup:** Attach the FRDM33771CSPEVB board to the MCU board.
   For TPL setup: Attach the FRDMDUAL33664EVB board to the MCU board and connect RD33771CDSTEVB board to the first TPL interface of FRDMDUAL33664EVB board (J1 connector) by a twisted pair cable.

3. Setup the battery pack emulator according to its user manual and connect it to the BCC board using a parallel cable.
4. Connect USB cable to USB interface on the MCU board and to your PC.

Figure 2 shows the complete HW setup for the TPL communication with MC33771C.

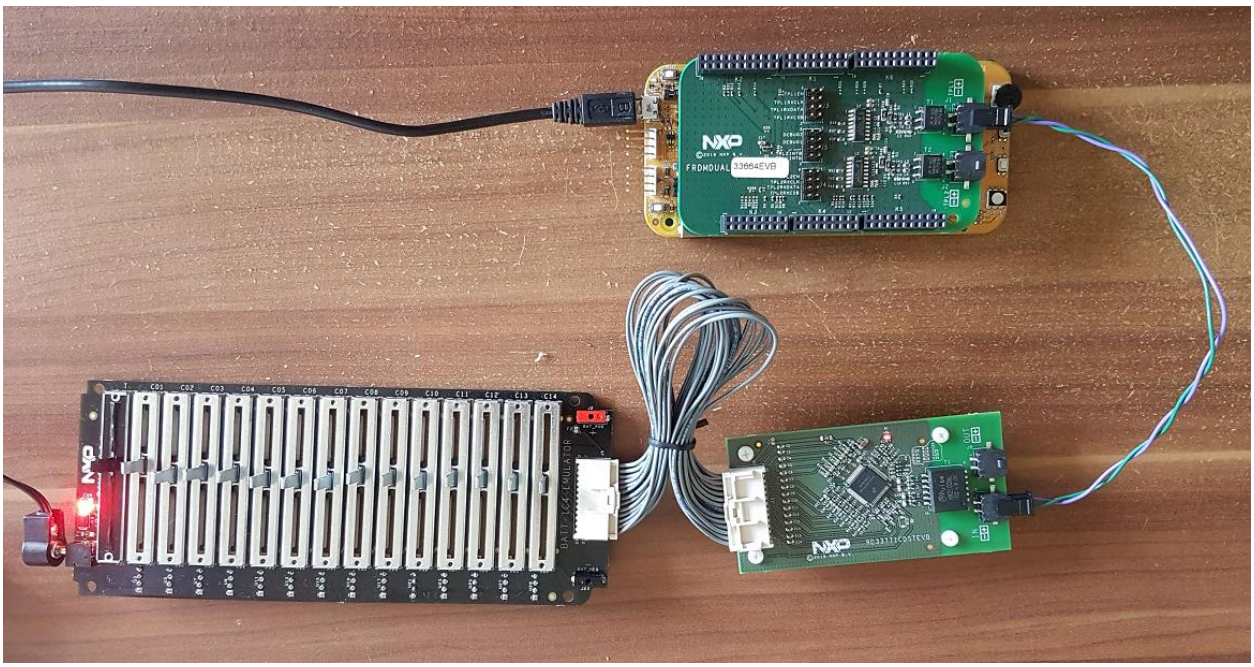

Figure 2. TPL HW setup

Setting up Software

Make sure that you have installed S32 Design Studio for ARM Version 2018.R1 with Update 11.

Description

The purpose of this example is to show how can be **Battery Cell Controller SW Driver for MC33771C** used in conjunction with **FreeMASTER visualization tool**.

Designed GUI (see Figure 3) provides means of control of the single BCC device. All of the main functions are covered, such as **monitoring** and **controlling** of the device.

- User can directly control the device mode (sleep, normal), software reset, cell balancing with timeout selection, clear all faults, set thresholds as input for supervision on OC, OT, UT, OV, UV.
- GUI also provides feedback from the BCC device in terms of measurements and fault status information.

Note that this application aims to **demonstrate** capabilities of the BCC device. It is not intended to cover the complex functionality which is provided by API functions of the BCC SW driver.

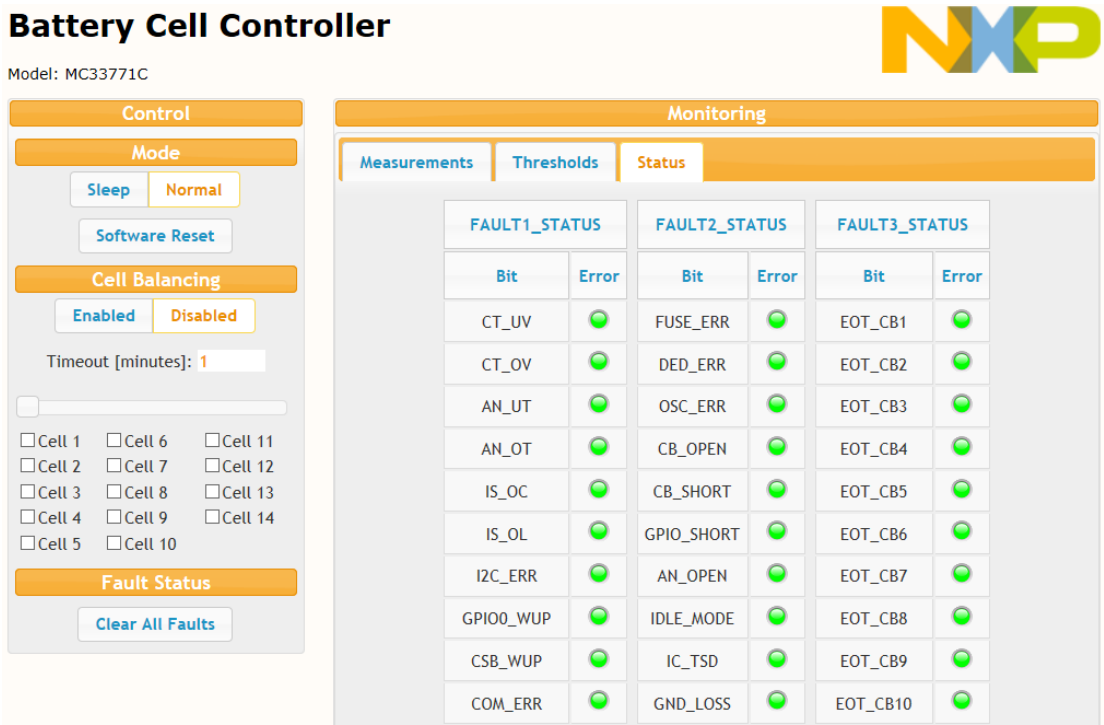

Figure 3. FreeMASTER application running on PC.

Application code initializes BCC SW Driver and then periodically repeats following tasks with specified timeout delay:

1. FreeMASTER – the task controls BCC device based on FreeMASTER commands.
2. Measurement task (each 200 ms) – the task reads measured values by the BCC device.
3. Status task (each 200 ms) – the task reads status (fault information) of the BCC device.

Application utilizes red LED on the MCU board to signalize any application error.

The project uses the following peripherals for **SPI setup**:

- LPSPI0 – SPI communication
- GPIO – RESET pin and red onboard LED
- LPUART1 – serial communication
- LPIT0 – Example project timing, timeout for BCC SW driver

The project uses the following peripherals for **TPL setup**:

- LPSPI0 – SPI RX communication
- LPSPI1 – SPI TX communication
- GPIO – INTB pin, EN pin, TPL1+2TXCSB pin and red onboard LED
- LPUART1 – serial communication
- LPIT0 – Example project timing, timeout for BCC SW driver

Pin selection for all mentioned peripherals is listed in Table 1 (SPI) and Table 2 (TPL).

Table 1. Pin selection for the SPI setup

Pin Function	S32K144
MISO	PTB3/LPSPI0_SIN
MOSI	PTB4/LPSPI0_SOUT
SCLK	PTB2/LPSPI0_SCK
CSB	PTB5
RESET	PTD4
Red on-board LED	PTD15
LPUART_RX	PTC6/LPUART1_RX
LPUART_TX	PTC7/LPUART1_TX

Table 2. Pin selection for the TPL setup

Pin Function	S32K144 pin
TPL1TXDATA	PTB16/LPSPI1_SOUT
TPL1TXSCLK	PTB14/LPSPI1_SCK
TPL1TXCSB	PTA6/LPSPI1_PCS1
TPL1+2TXCSB	PTD3
TPL1RXDATA	PTB3/LPSPI0_SIN
TPL1RXSCLK	PTB2/LPSPI0_SCK
TPL1RXCSB	PTB5/LPSPI0_PCS1
TPL1EN	PTB0
TPL1INTB	PTB1
Red on-board LED	PTD15
LPUART_RX	PTC6/LPUART1_RX
LPUART_TX	PTC7/LPUART1_TX

As you can see in Figure 4, example project consists of several files/folders apart from main.c.

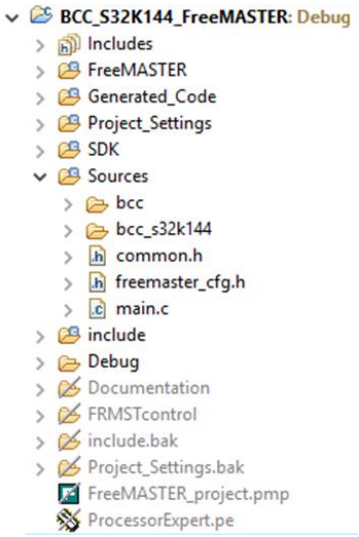

Figure 4. Example Code Structure

- **FreeMASTER** – FreeMASTER driver for S32K144.
- **Sources/bcc** contains the BCC SW driver for MC33771C.
- **Sources/bcc_s32k144** contains external functions of BCC SW driver related to this example and S32K144 MCU.
- **Sources/freemaster_cfg.c** is configuration of FreeMASTER driver.
- **Sources/main.c** contains functions for setup of used MCU peripherals and functions implementing functionality described above.
- **Sources/common.h** contains definitions reflecting the HW setup as initial BCC device register values.

Additionally configuration of several used MCU peripherals is done by Processor Expert components. Outputs of those are generated configuration structures (*Generated_Code* folder) which are then passed to initialization functions in *main.c*.

Import the Example Project

The following steps shows how to import an example project into S32 Design Studio for ARM Version 2018.R1.

1. In S32 DS click on the ***File / Import***.
2. Choose ***General / Existing Projects into Workspace***.
3. Click ***Browse to select root directory*** with your downloaded example projects.
4. ***Select project*** named **BCC_S32K144_FreeMASTER** and click ***Finish*** to complete the process.
5. At the top of **Sources/common.h** file define either macro **SPI** (in case of SPI HW setup) or **TPL** (in case of TPL HW setup).
6. Now the example project should be in your workspace and ready to run.

Building and Running the Project

In order to build and run the project you need to ***build*** the project in a usual way. If the build is successful, ***debug and run*** the project. This can be accomplished in following steps:

1. Click on the **arrow** next to the **debug icon** and select **Debug Configurations**.
2. **Select** one of the existing configurations with **project name** under **PEMicro** group or **create** one by double clicking on this group.
3. Pick up proper **debug interface** and **USB port**.
4. Apply changes and click on **Debug**.

Note that the first build shows several warnings related to implicit declaration of math functions (Figure 5). These warnings have no impact on example functionality and will disappear after next build.

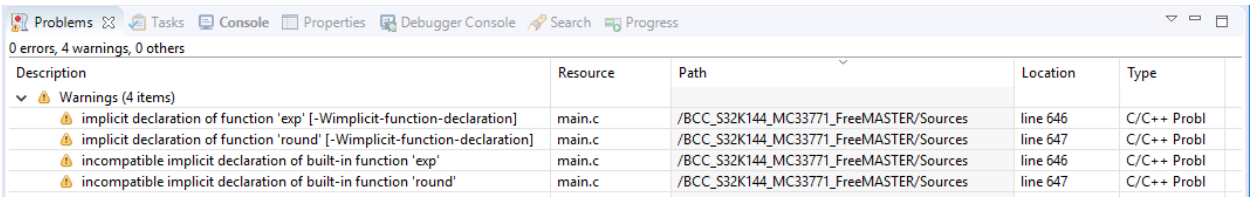

Figure 5. Warnings after the first build

If you have any questions related to how to work with debug configurations, see ***S32 Design Studio User's Guide***.

In order to run the FreeMASTER application, instructions follows:

1. Launch FreeMASTER application. You can download it from NXP webpage. Install FreeMASTER by clicking downloaded **FMASTERSW.exe** application.
2. Click **File / Open Project…** (see Figure 6).

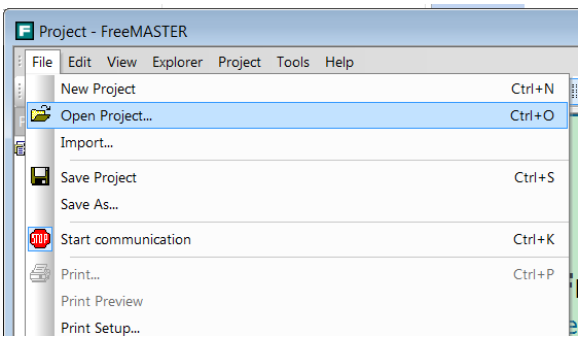

Figure 6. Opening FreeMASTER application

3. Locate FreeMASTER project file (.pmp) in the example project folder: BCC_SDK_SW\\S32DS_Examples\BCC_S32K144_FreeMASTER. Select **FreeMASTER_project.pmp** file then click Open (see Figure 7).
    - If a dialog saying Missing symbol definition pops up click Continue. It is needed to link .elf file to the FreeMASTER project. Click **Project->Options** and under Map Files tab there is **Default symbol** file input. Select .elf file from Debug directory which is located in this BCC_S32K144_FreeMASTER project folder. If the .elf file is not present, you have to build the project in S32DS.

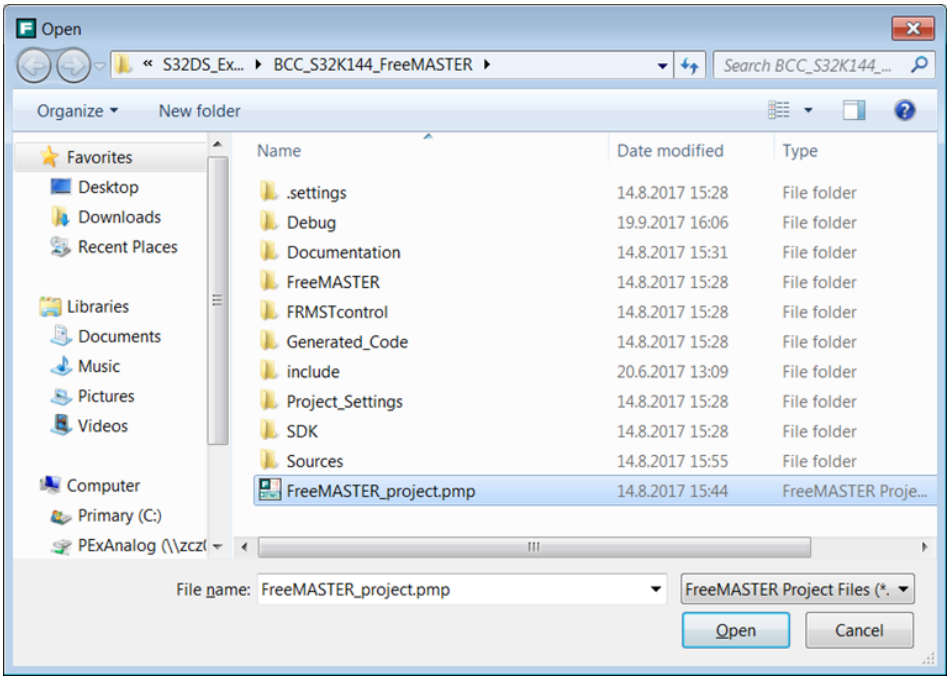

Figure 7. Selection of FreeMASTER project

4. Check if the correct COM port is selected. Click **Project->Options**. Check if the number of the COM port in FreeMASTER corresponds to the number of COM port on your computer (see Figure 8). Also check if the communication speed corresponds to the UART communication speed implemented in the MCU firmware (115 200 bauds).

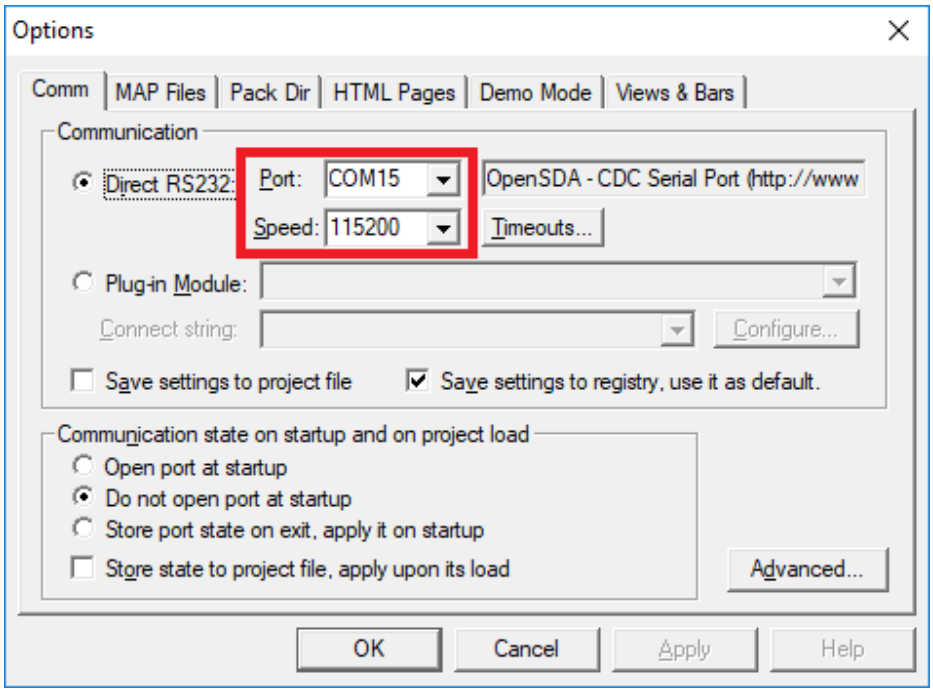

Figure 8. FreeMASTER settings

5. Start FreeMASTER monitoring (click on the red button named **STOP**, see Figure 9).

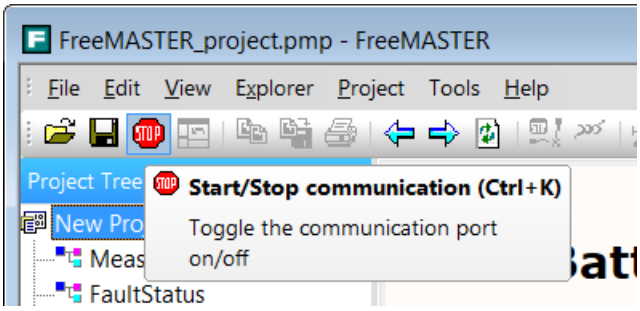

Figure 9. Run FreeMASTER application